Exercícios Avaliativos

Os alunos deverão modificar as classes, adicionando novas funcionalidades enquanto mantêm o comportamento original da estrutura de dados.

📌 Exercício 1: Fila com Contador de Operações (Queue)

Desafio: Modifique a classe Queue para contar quantas operações foram feitas na fila.

Regras:

1. Crie uma variável privada #operationCount para contar quantas operações foram realizadas (enqueue e dequeue).
2. Adicione um método getOperationCount() que retorna o total de operações feitas.
3. Teste essa funcionalidade simulando uma fila de atendimento ao cliente.

📌 Exercício 2: Pilha com Histórico de Remoções (Stack)

Desafio: Modifique a classe Stack para armazenar um histórico dos elementos que foram removidos (pop).

Regras:

1. Crie um **array privado** #historyData para armazenar os elementos removidos.
2. Adicione um método history() para retornar os elementos que já foram retirados da pilha.
3. Teste essa funcionalidade simulando uma pilha de navegação em um navegador (ex: páginas visitadas e removidas).

📌 Exercício 3: Deque com Capacidade Limitada

Desafio: Modifique a classe Deque para definir um **tamanho máximo** e impedir que novos elementos sejam inseridos quando o limite for atingido.

Regras:

1. O tamanho máximo da deque deve ser passado no **construtor**.
2. Se o limite for atingido, os métodos insertFront() e insertBack() devem exibir uma **mensagem de erro** e impedir a inserção.
3. Teste essa funcionalidade simulando um deque com capacidade de apenas 5 elementos.

📌 Exercício 4: Pilha com Capacidade Máxima

Desafio: Modifique a classe Stack para ter um **limite máximo de elementos**.

Regras:

1. O tamanho máximo da pilha deve ser definido no **construtor**.
2. Se a pilha atingir o limite, o método push() deve exibir uma **mensagem de erro** e impedir a inserção.
3. Teste essa funcionalidade simulando uma pilha de **desfazer ações** em um editor de texto.

📌 Exercício 5: Deque com Elementos Únicos

Desafio: Modifique a classe Deque para não permitir **valores duplicados**.

Regras:

1. Antes de adicionar um novo valor (insertFront ou insertBack), verifique se ele **já está na deque**.
2. Se o valor já existir, exiba uma **mensagem de erro** e ignore a inserção.
3. Teste essa funcionalidade simulando um deque de **itens únicos em um carrinho de compras**.